

C++

```
void App::OnCapturingChanged(Windows::Media::Capture::AppCapture^ sender,
Platform::Object^ args)
{
    Platform::String^ captureStatusText = "";
    if (sender->IsCapturingAudio)
    {
        captureStatusText += "Capturing audio.";
    }
    if (sender->IsCapturingVideo)
    {
        captureStatusText += "Capturing video.";
    }
    UpdateStatusText(captureStatusText);
}
```

Add the Windows Desktop Extensions for the UWP to your app

The APIs for recording audio and video and for capturing screenshots directly from your app, found in the [Windows.Media.AppRecording](#) namespace, are not included in the Universal API contract. To access the APIs, you must add a reference to the Windows Desktop Extensions for the UWP to your app with the following steps.

1. In Visual Studio, in **Solution Explorer**, expand your UWP project and right-click **References** and then select **Add Reference....**
2. Expand the **Universal Windows** node and select **Extensions**.
3. In the list of extensions check the checkbox next to the **Windows Desktop Extensions for the UWP** entry that matches the target build for your project. For the app broadcast features, the version must be 1709 or greater.
4. Click **OK**.

Get an instance of AppRecordingManager

The [AppRecordingManager](#) class is the central API you will use to manage app recording. Get an instance of this class by calling the factory method [GetDefault](#). Before using any of the APIs in the **Windows.Media.AppRecording** namespace, you should check for their presence on the current device. The APIs are not available on devices running an OS version earlier than Windows 10, version 1709. Rather than check for a specific OS version, use the [ApiInformation.IsApiContractPresent](#) method to query for the *Windows.Media.AppBroadcasting.AppRecordingContract* version 1.0. If this contract is present, then the recording APIs are available on the device. The example code in this

article checks for the APIs once and then checks if the **AppRecordingManager** is null before subsequent operations.

C++

```
if (Windows::Foundation::Metadata::ApiInformation::IsApiContractPresent(
    "Windows.Media.AppRecording.AppRecordingContract", 1, 0))
{
    m_appRecordingManager = AppRecordingManager::GetDefault();
}
```

Determine if your app can currently record

There are several reasons that your app may not currently be able to capture audio or video, including if the current device doesn't meet the hardware requirements for recording or if another app is currently broadcasting. Before initiating a recording, you can check to see if your app is currently able to record. Call the **GetStatus** method of the **AppRecordingManager** object and then check the **CanRecord** property of the returned **AppRecordingStatus** object. If **CanRecord** returns **false**, meaning that your app can't currently record, you can check the **Details** property to determine the reason. Depending on the reason, you may want to display the status to the user or show instructions for enabling app recording.

C++

```
bool App::CanRecord()
{
    if (m_appRecordingManager == nullptr)
    {
        return false;
    }

    AppRecordingStatus^ recordingStatus = m_appRecordingManager-
>GetStatus();

    if (!recordingStatus->CanRecord)
    {
        AppRecordingStatusDetails^ details = recordingStatus->Details;

        if (details->IsAnyAppBroadcasting)
        {
            UpdateStatusText("Another app is currently broadcasting.");
            return false;
        }

        if (details->IsCaptureResourceUnavailable)
        {

```

```

        UpdateStatusText("The capture resource is currently
unavailable.");
        return false;
    }

    if (details->IsGameStreamInProgress)
    {
        UpdateStatusText("A game stream is currently in progress.");
        return false;
    }

    if (details->IsGpuConstrained)
    {
        // Typically, this means that the GPU software does not include
an H264 encoder
        UpdateStatusText("The GPU does not support app recording.");
        return false;
    }

    if (details->IsAppInactive)
    {
        // Broadcasting can only be started when the application's
window is the active window.
        UpdateStatusText("The app window to be recorded is not
active.");
        return false;
    }

    if (details->IsBlockedForApp)
    {
        UpdateStatusText("Recording is blocked for this app.");
        return false;
    }

    if (details->IsDisabledByUser)
    {
        UpdateStatusText("The user has disabled GameBar in Windows
Settings.");
        return false;
    }

    if (details->IsDisabledBySystem)
    {
        UpdateStatusText("Recording is disabled by the system.");
        return false;
    }

    return false;
}

return true;
}

```

Manually start and stop recording your app to a file

After verifying that your app is able to record, you can start a new recording by calling the [StartRecordingToFileAsync](#) method of the **AppRecordingManager** object.

In the following example, the first **then** block executes when the asynchronous task fails. The second **then** block attempts to access the result of the task and, if the result is null, then the task has completed. In both cases, the **OnRecordingComplete** helper method, shown below, is called to handle the result.

C++

```
void App::StartRecordToFile(Windows::Storage::StorageFile^ file)
{
    if (m_appRecordingManager == nullptr)
    {
        return;
    }

    if (!CanRecord())
    {
        return;
    }

    // Start a recording operation to record starting from
    // now until the operation fails or is cancelled.
    m_recordOperation = m_appRecordingManager->StartRecordingToFileAsync(file);

    create_task(m_recordOperation).then(
        [this](AppRecordingResult^ result)
    {
        OnRecordingComplete();
    }).then([this](task<void> t)
    {
        try
        {
            t.get();
        }
        catch (const task_canceled&)
        {
            OnRecordingComplete();
        }
    })
```

```
});
}
```

When the recording operation completes, check the [Succeeded](#) property of the returned [AppRecordingResult](#) object to determine if the record operation was successful. If so, you can check the [IsFileTruncated](#) property to determine if, for storage reasons, the system was forced to truncate the captured file. You can check the [Duration](#) property to discover the actual duration of the recorded file which, if the file is truncated, may be shorter than the duration of the recording operation.

C++

```
void App::OnRecordingComplete()
{
    if (m_recordOperation)
    {
        auto result = m_recordOperation->GetResults();

        if (result->Succeeded)
        {
            Windows::Foundation::TimeSpan duration = result->Duration;
            boolean isTruncated = result->IsFileTruncated;

            UpdateStatusText("Recording completed.");
        }
        else
        {
            // If the recording failed, ExtendedError
            // can be retrieved and used for diagnostic purposes
            HRESULT extendedError = result->ExtendedError;
            LogTelemetryMessage("Error during recording: " + extendedError);
        }

        m_recordOperation = nullptr;
    }
}
```

The following examples show some basic code for starting and stopping the recording operation shown in the previous example.

C++

```
StorageFolder^ storageFolder = ApplicationData::Current->LocalFolder;
concurrency::create_task(storageFolder-
>CreateFileAsync("recordtofile_example.mp4",
CreationCollisionOption::ReplaceExisting)).then(
    [this](StorageFile^ file)
{
    }
```

```
        StartRecordToFile(file);
    });
```

C++

```
void App::FinishRecordToFile()
{
    m_recordOperation->Cancel();
}
```

Record a historical time span to a file

If the user has enabled historical recording for your app in the system settings, you can record a time span of gameplay that has previously transpired. A previous example in this article showed how to confirm that your app can currently record gameplay. There is an additional check to determine if historical capture is enabled. Once again, call [GetStatus](#) and check the [CanRecordTimeSpan](#) property of the returned **AppRecordingStatus** object. This example also returns the [HistoricalBufferDuration](#) property of the **AppRecordingStatus** which will be used to determine a valid start time for the recording operation.

C++

```
bool App::CanRecordTimeSpan(TimeSpan &historicalDurationBuffer)
{
    if (m_appRecordingManager == nullptr)
    {
        return false;
    }

    AppRecordingStatus^ recordingStatus = m_appRecordingManager-
>GetStatus();
    if (recordingStatus->Details->IsTimeSpanRecordingDisabled)
    {
        UpdateStatusText("Historical time span recording is disabled by the system.");
        return false;
    }

    historicalDurationBuffer = recordingStatus->HistoricalBufferDuration;

    return true;
}
```

To capture a historical timespan, you must specify a start time for the recording and a duration. The start time is provided as a [DateTime](#) struct. The start time must be a time before the current time, within the length of the historical recording buffer. For this example, the buffer length is retrieved as part of the check to see if historical recording is enabled, which is shown in the previous code example. The duration of the historical recording is provided as [TimeSpan](#) struct, which should also be equal to or smaller than the duration of the historical buffer. Once you have determined your desired start time and duration, call [RecordTimeSpanToFileAsync](#) to start the recording operation.

Like recording with manual start and stop, when a historical recording completes, you can check the [Succeeded](#) property of the returned [AppRecordingResult](#) object to determine if the record operation was successful, and you can check the [IsFileTruncated](#) and [Duration](#) property to discover the actual duration of the recorded file which, if the file is truncated, may be shorter than the duration of the requested time window.

C++

```
void App::RecordTimeSpanToFile(Windows::Storage::StorageFile^ file)
{

    if (m_appRecordingManager == nullptr)
    {
        return;
    }

    if (!CanRecord())
    {
        return;
    }

    Windows::Foundation::TimeSpan historicalBufferDuration;
    if (!CanRecordTimeSpan(historicalBufferDuration))
    {
        return;
    }

    AppRecordingStatus^ recordingStatus = m_appRecordingManager-
>GetStatus();

    Windows::Globalization::Calendar^ calendar = ref new
Windows::Globalization::Calendar();
    calendar->SetToNow();

    Windows::Foundation::DateTime nowTime = calendar->GetDateTime();

    int secondsToRecord = min(30, historicalBufferDuration.Duration /
10000000);
    calendar->AddSeconds(-1 * secondsToRecord);
```

```

Windows::Foundation::DateTime startTime = calendar->GetDateTime();

Windows::Foundation::TimeSpan duration;

duration.Duration = nowTime.UniversalTime - startTime.UniversalTime;

create_task(m_appRecordingManager->RecordTimeSpanToFileAsync(startTime,
duration, file)).then(
    [this](AppRecordingResult^ result)
    {
        if (result->Succeeded)
        {
            Windows::Foundation::TimeSpan duration = result->Duration;
            boolean isTruncated = result->IsFileTruncated;
            UpdateStatusText("Recording completed.");
        }
        else
        {
            // If the recording failed, ExtendedError
            // can be retrieved and used for diagnostic purposes
            HRESULT extendedError = result->ExtendedError;
            LogTelemetryMessage("Error during recording: " + extendedError);
        }
    });
}

```

The following example shows some basic code for initiating the historical record operation shown in the previous example.

C++

```

StorageFolder^ storageFolder = ApplicationData::Current->LocalFolder;
concurrency::create_task(storageFolder->CreateFileAsync("recordtimespantofile_example.mp4",
CreationCollisionOption::ReplaceExisting)).then(
    [this](StorageFile^ file)
    {
        RecordTimeSpanToFile(file);
    });

```

Save screenshot images to files

Your app can initiate a screenshot capture that will save the current contents of the app's window to one image file or to multiple image files with different image encodings. To specify the image encodings you would like to use, create a list of strings where each represents an image type. The properties of the [ImageEncodingSubtypes](#)

provide the correct string for each supported image type, such as **MediaEncodingSubtypes.Png** or **MediaEncodingSubtypes.JpegXr**.

Initiate screen capture by calling the **SaveScreenshotToFilesAsync** method of the **AppRecordingManager** object. The first parameter to this method is a **StorageFolder** where the image files will be saved. The second parameter is a filename prefix to which the system will append the extension for each image type saved, such as ".png".

The third parameter to **SaveScreenshotToFilesAsync** is necessary for the system to be able to do the proper colorspace conversion if the current window to be captured is displaying HDR content. If HDR content is present, this parameter should be set to **AppRecordingSaveScreenshotOption.HdrContentVisible**. Otherwise, use **AppRecordingSaveScreenshotOption.None**. The final parameter to the method is the list of image formats to which the screen should be captured.

When the asynchronous call to **SaveScreenshotToFilesAsync** completes, it returns a **AppRecordingSavedScreenshotInfo** object that provides the **StorageFile** and associated **MediaEncodingSubtypes** value indicating the image type for each saved image.

C++

```
void App::SaveScreenShotToFiles(Windows::Storage::StorageFolder^ folder,
Platform::String^ filenamePrefix)
{
    if (m_appRecordingManager == nullptr)
    {
        return;
    }

    Windows::Foundation::Collections::IVectorView<Platform::String^>^
supportedFormats =
    m_appRecordingManager->SupportedScreenshotMediaEncodingSubtypes;

    Platform::Collections::Vector<Platform::String^>^ requestedFormats =
    ref new Platform::Collections::Vector<Platform::String^>();

    for (Platform::String^ format : requestedFormats)
    {
        if (format ==
Windows::Media::MediaProperties::MediaEncodingSubtypes::Png)
        {
            requestedFormats->Append(format);
        }
        else if (format ==
Windows::Media::MediaProperties::MediaEncodingSubtypes::JpegXr)
        {
            requestedFormats->Append(format);
        }
    }
}
```

```

    }
}

create_task(m_appRecordingManager->SaveScreenshotToFilesAsync(folder,
filenamePrefix, AppRecordingSaveScreenshotOption::None,
requestedFormats->GetView())).then(
    [this](AppRecordingSaveScreenshotResult^ result)
    {
        if (result->Succeeded)
        {

Windows::Foundation::Collections::IVectorView<AppRecordingSavedScreenshotInfo>^ returnedScreenshots = result->SavedScreenshotInfos;

            for (AppRecordingSavedScreenshotInfo^ screenshotInfo :
returnedScreenshots)
            {
                Windows::Storage::StorageFile^ file = screenshotInfo->File;
                Platform::String^ type = screenshotInfo->
>MediaEncodingSubtype;
            }
        }
        else
        {
            // If the recording failed, ExtendedError
            // can be retrieved and used for diagnostic purposes
            HRESULT extendedError = result->ExtendedError;
            LogTelemetryMessage("Error during screenshot: " +
extendedError);
        }
    });
}

```

The following example shows some basic code for initiating the screenshot operation shown in the previous example.

C++

```

StorageFolder^ storageFolder = ApplicationData::Current->LocalFolder;
SaveScreenShotToFiles(storageFolder, "screen_capture");

```

Add game metadata for system and app-initiated capture

The following sections of this article describe how to provide metadata that the system will embed into the MP4 stream of captured or broadcast gameplay. Metadata can be embedded in media that is captured using the built-in system UI and media that is

captured by the app with **AppRecordingManager**. This metadata can be extracted by your app and other apps during media playback in order to provide contextually-aware experiences that are synchronized with the captured or broadcast gameplay.

Get an instance of AppCaptureMetadataWriter

The primary class for managing app capture metadata is **AppCaptureMetadataWriter**. Before initializing an instance of this class, use the **ApiInformation.IsApiContractPresent** method to query for the *Windows.Media.Capture.AppCaptureMetadataContract* version 1.0 to verify that the API is available on the current device.

C++

```
if
(Windows::Foundation::Metadata::ApiInformation::IsApiContractPresent("Windows.Media.Capture.AppCaptureMetadataContract", 1, 0))
{
    m_appCaptureMetadataWriter = ref new AppCaptureMetadataWriter();
}
```

Write metadata to the system cache for your app

Each metadata item has a string label, identifying the metadata item, an associated data value which can be a string, an integer, or a double value, and a value from the **AppCaptureMetadataPriority** enumeration indicating the relative priority of the data item. A metadata item can either be considered an "event", which occurs at a single point in time, or a "state" which maintains a value over a time window. Metadata is written to a memory cache that is allocated and managed for your app by the system. The system enforces a size limit on the metadata memory cache and, when the limit is reached, will purge data based on the priority with which each metadata item was written. The next section of this article shows how to manage your app's metadata memory allocation.

A typical app may choose to write some metadata at the beginning of the capture session to provide some context for the subsequent data. For this scenario it is recommended that you use instantaneous "event" data. This example calls **AddStringEvent**, **AddDoubleEvent**, and **AddInt32Event** to set instantaneous values for each data type.

C++

```
void App::StartSession(Platform::String^ sessionId, double averageFps, int resolutionWidth, int resolutionHeight)
```

```

{
    if (m_appCaptureMetadataWriter != nullptr)
    {
        m_appCaptureMetadataWriter->AddStringEvent("sessionId", sessionId,
AppCaptureMetadataPriority::Informational);
        m_appCaptureMetadataWriter->AddDoubleEvent("averageFps", averageFps,
AppCaptureMetadataPriority::Informational);
        m_appCaptureMetadataWriter->AddInt32Event("resolutionWidth",
resolutionWidth, AppCaptureMetadataPriority::Informational);
        m_appCaptureMetadataWriter->AddInt32Event("resolutionHeight",
resolutionHeight, AppCaptureMetadataPriority::Informational);
    }
}

```

A common scenario for using "state" data that persists over time is to track the game map that the player is currently within. This example calls [StartStringState](#) to set the state value.

C++

```

void App::StartMap(Platform::String^ mapName)
{
    m_appCaptureMetadataWriter->StartStringState("map", mapName,
AppCaptureMetadataPriority::Important);
}

```

Call [StopState](#) to record that a particular state has ended.

C++

```

void App::EndMap(Platform::String^ mapName)
{
    m_appCaptureMetadataWriter->StopState("map");
}

```

You can overwrite a state by setting a new value with an existing state label.

C++

```

void App::LevelUp(int newLevel)
{
    m_appCaptureMetadataWriter->StartInt32State("currentLevel", newLevel,
AppCaptureMetadataPriority::Important);
}

```

You can end all currently open states by calling [StopAllStates](#).

C++

```
void App::RaceComplete()
{
    m_appCaptureMetadataWriter->StopAllStates();
}
```

Manage metadata cache storage limit

The metadata that you write with **AppCaptureMetadataWriter** is cached by the system until it is written to the associated media stream. The system defines a size limit for each app's metadata cache. Once the cache size limit has been reached, the system will begin purging cached metadata. The system will delete metadata that was written with **AppCaptureMetadataPriority.Informational** priority value before deleting metadata with the **AppCaptureMetadataPriority.Important** priority.

At any point, you can check to see the number of bytes available in your app's metadata cache by calling **RemainingStorageBytesAvailable**. You can choose to set your own app-defined threshold after which you can choose to reduce the amount of metadata that you write to the cache. The following example shows a simple implementation of this pattern.

C++

```
void App::CheckMetadataStorage()
{
    INT64 storageRemaining = m_appCaptureMetadataWriter-
>RemainingStorageBytesAvailable;

    if (storageRemaining < m_myLowStorageLevelInBytes)
    {
        m_writeLowPriorityMetadata = false;
    }
}
```

C++

```
void App::ComboExecuted(Platform::String^ comboName)
{
    if (m_writeLowPriorityMetadata)
    {
        m_appCaptureMetadataWriter->AddStringEvent("combo", comboName,
AppCaptureMetadataPriority::Informational);
    }
}
```

Receive notifications when the system purges metadata

You can register to receive a notification when the system begins purging metadata for your app by registering a handler for the [MetadataPurged](#) event.

C++

```
if (m_appCaptureMetadataWriter != nullptr)
{
    m_appCaptureMetadataWriter->MetadataPurged +=
        ref new TypedEventHandler<AppCaptureMetadataWriter^,
Platform::Object^>(this, &App::OnMetadataPurged);
}
```

In the handler for the **MetadataPurged** event, you can clear up some room in the metadata cache by ending lower-priority states, you can implement app-defined logic for reducing the amount of metadata you write to the cache, or you can do nothing and let the system continue to purge the cache based on the priority with which it was written.

C++

```
void
App::OnMetadataPurged(Windows::Media::Capture::AppCaptureMetadataWriter^
sender, Platform::Object^ args)
{
    // Reduce metadata by stopping a low-priority state.
    //m_appCaptureMetadataWriter->StopState("map");

    // Reduce metadata by stopping all states.
    //m_appCaptureMetadataWriter->StopAllStates();

    // Change app-specific behavior to write less metadata.
    //m_writeLowPriorityMetadata = false;

    // Take no action. Let the system purge data as needed. Record event for
    telemetry.
    OutputDebugString(TEXT("Low-priority metadata purged."));
}
```

Related topics

- [Gaming](#)

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Input for games

Article • 10/20/2022

This section describes the different kinds of input devices that can be used in Universal Windows Platform (UWP) games on Windows 10 or Windows 11 and Xbox, demonstrates their basic usage, and recommends patterns and techniques for effective input programming in games.

ⓘ Note

Other kinds of input devices exist and are available to be used in UWP games such as custom input devices that might be genre-specific or game-specific. Such devices and their programming are not discussed in this section. For information on the interfaces used to facilitate custom input devices, see the [Windows.Gaming.Input.Custom](#) namespace.

Gaming input devices

Game input devices are supported in UWP games and apps for Windows 10 or Windows 11 and Xbox One through the [Windows.Gaming.Input](#) namespace.

Gamepads

Gamepads are the standard input device on Xbox and a common choice for Windows gamers when they don't favor a keyboard and mouse. They provide a variety of digital and analog controls making them suitable for almost any kind of game and also provide tactile feedback through embedded vibration motors.

For information on how to use gamepads in your UWP game, see [Gamepad and vibration](#).

Arcade sticks

Arcade sticks are all-digital input devices valued for reproducing the feel of stand-up arcade machines and are the perfect input device for head-to-head-fighting or other arcade-style games.

For information on how to use arcade sticks in your UWP game, see [Arcade stick](#).

Racing wheels

Racing wheels are input devices that resemble the feel of a real racecar cockpit and are the perfect input device for any racing game that features cars or trucks. Many racing wheels are equipped with true force feedback--that is, they can apply actual forces on an axis of control such as the steering wheel--not just simple vibration.

For information on how to use racing wheels in your UWP game, see [Racing Wheel and force feedback](#).

Flight sticks

Flight sticks are gaming input devices that reproduce the feel of flight sticks that would be found in a plane or spaceship's cockpit. They're the perfect input device for quick and accurate control of flight.

For more information on how to use flight sticks in your UWP game, see [Flight stick](#).

Raw game controllers

A raw game controller is a generic representation of a game controller, with inputs found on many different kinds of common game controllers. These inputs are exposed as simple arrays of unnamed buttons, switches, and axes. Using a raw game controller, you can allow customers to create custom input mappings no matter what type of controller they're using.

For more information on how to use raw game controllers in your UWP game, see [Raw game controller](#).

UI navigation controllers

UI Navigation controllers are a logical input device that exists to provide a common vocabulary for UI navigation commands that promotes a consistent user experience across different games and physical input devices. A game's user interface should use the UINavigationController interfaces instead of device-specific interfaces.

For information on how to use UI navigation controllers in your UWP game, see [UI navigation controller](#).

Headsets

Headsets are audio capture and playback devices that are associated with a specific user when connected through their input device. They're commonly used by online games for voice chat but can also be used to enhance immersion or provide gameplay features in both online and offline games.

For information on how to use headsets in your UWP game, see [Headset](#)

Users

Each input device and its connected headset can be associated with a specific user to link their identity to their gameplay. The user identity is also the means by which input from a physical input device is correlated to input from its logical UI navigation controller.

For information on how to manage users and their input devices, see [Tracking users and their devices](#).

See Also

- [Input practices for games](#)
- [Windows.Gaming.Input namespace](#)
- [Windows.Gaming.Input.Custom namespace](#)

Gamepad and vibration

Article • 10/20/2022

This page describes the basics of programming for game pads using `[Windows.Gaming.Input.Gamepad][game pad]` and related APIs for the Universal Windows Platform (UWP).

By reading this page, you'll learn:

- how to gather a list of connected game pads and their users
- how to detect that a game pad has been added or removed
- how to read input from one or more game pads
- how to send vibration and impulse commands
- how game pads behave as UI navigation devices

Gamepad overview

Gamepads like the Xbox Wireless Controller and Xbox Wireless Controller S are general-purpose gaming input devices. They're the standard input device on Xbox One and a common choice for Windows gamers when they don't favor a keyboard and mouse. Gamepads are supported in Windows 10 or Windows 11 and Xbox UWP apps through the [Windows.Gaming.Input](#) namespace.

Xbox One gamepads are equipped with a directional pad (or D-pad); **A**, **B**, **X**, **Y**, **View**, and **Menu** buttons; left and right thumbsticks, bumpers, and triggers; and a total of four vibration motors. Both thumbsticks provide dual analog readings in the X and Y axes, and also act as a button when pressed inward. Each trigger provides an analog reading that represents how far it's pulled back.

ⓘ Note

`Windows.Gaming.Input.Gamepad` also supports Xbox 360 gamepads, which have the same control layout as standard Xbox One gamepads.

Vibration and impulse triggers

Xbox One gamepads provide two independent motors for strong and subtle gamepad vibration as well as two dedicated motors for providing sharp vibration to each trigger (this unique feature is the reason that Xbox One gamepad triggers are referred to as *impulse triggers*).

ⓘ Note

Xbox 360 gamepads are not equipped with *impulse triggers*.

For more information, see [Vibration and impulse triggers overview](#).

Thumbstick deadzones

A thumbstick at rest in the center position would ideally produce the same, neutral reading in the X and Y axes every time. However, due to mechanical forces and the sensitivity of the thumbstick, actual readings in the center position only approximate the ideal neutral value and can vary between subsequent readings. For this reason, you must always use a small *deadzone*—a range of values near the ideal center position that are ignored—to compensate for manufacturing differences, mechanical wear, or other gamepad issues.

Larger deadzones offer a simple strategy for separating intentional input from unintentional input.

For more information, see [Reading the thumbsticks](#).

UI navigation

In order to ease the burden of supporting the different input devices for user interface navigation and to encourage consistency between games and devices, most *physical* input devices simultaneously act as a separate *logical* input device called a [UI navigation controller](#). The UI navigation controller provides a common vocabulary for UI navigation commands across input devices.

As a UI navigation controller, gamepads map the [required set](#) of navigation commands to the left thumbstick, D-pad, **View**, **Menu**, **A**, and **B** buttons.

Navigation command	Gamepad input
Up	Left thumbstick up / D-pad up
Down	Left thumbstick down / D-pad down
Left	Left thumbstick left / D-pad left
Right	Left thumbstick right / D-pad right
View	View button

Navigation command	Gamepad input
Menu	Menu button
Accept	A button
Cancel	B button

Additionally, gamepads map all of the [optional set](#) of navigation commands to the remaining inputs.

Navigation command	Gamepad input
Page Up	Left trigger
Page Down	Right trigger
Page Left	Left bumper
Page Right	Right bumper
Scroll Up	Right thumbstick up
Scroll Down	Right thumbstick down
Scroll Left	Right thumbstick left
Scroll Right	Right thumbstick right
Context 1	X Button
Context 2	Y Button
Context 3	Left thumbstick press
Context 4	Right thumbstick press

Detect and track gamepads

Gamepads are managed by the system, therefore you don't have to create or initialize them. The system provides a list of connected gamepads and events to notify you when a gamepad is added or removed.

The gamepads list

The [Gamepad](#) class provides a static property, [Gamepads](#), which is a read-only list of gamepads that are currently connected. Because you might only be interested in some

of the connected gamepads, it's recommended that you maintain your own collection instead of accessing them through the `Gamepads` property.

The following example copies all connected gamepads into a new collection. Note that because other threads in the background will be accessing this collection (in the [GamepadAdded](#) and [GamepadRemoved](#) events), you need to place a lock around any code that reads or updates the collection.

C++

```
auto myGamepads = ref new Vector<Gamepad^>();
critical_section myLock{};

for (auto gamepad : Gamepad::Gamepads)
{
    // Check if the gamepad is already in myGamepads; if it isn't, add it.
    critical_section::scoped_lock lock{ myLock };
    auto it = std::find(begin(myGamepads), end(myGamepads), gamepad);

    if(it == end(myGamepads))
    {
        // This code assumes that you're interested in all gamepads.
        myGamepads->Append(gamepad);
    }
}
```

C#

```
private readonly object myLock = new object();
private List<Gamepad> myGamepads = new List<Gamepad>();
private Gamepad mainGamepad;

private void GetGamepads()
{
    lock (myLock)
    {
        foreach (var gamepad in Gamepad.Gamepads)
        {
            // Check if the gamepad is already in myGamepads; if it isn't,
            add it.
            bool gamepadInList = myGamepads.Contains(gamepad);

            if (!gamepadInList)
            {
                // This code assumes that you're interested in all gamepads.
                myGamepads.Add(gamepad);
            }
        }
    }
}
```

Adding and removing gamepads

When a gamepad is added or removed, the [GamepadAdded](#) and [GamepadRemoved](#) events are raised. You can register handlers for these events to keep track of the gamepads that are currently connected.

The following example starts tracking a gamepad that's been added.

C++

```
Gamepad::GamepadAdded += ref new EventHandler<Gamepad^>(Platform::Object^,
Gamepad^ args)
{
    // Check if the just-added gamepad is already in myGamepads; if it
    isn't, add
    // it.
    critical_section::scoped_lock lock{ myLock };
    auto it = std::find(begin(myGamepads), end(myGamepads), args);

    if (it == end(myGamepads))
    {
        // This code assumes that you're interested in all new gamepads.
        myGamepads->Append(args);
    }
}
```

C#

```
Gamepad.GamepadAdded += (object sender, Gamepad e) =>
{
    // Check if the just-added gamepad is already in myGamepads; if it
    isn't, add
    // it.
    lock (myLock)
    {
        bool gamepadInList = myGamepads.Contains(e);

        if (!gamepadInList)
        {
            myGamepads.Add(e);
        }
    }
};
```

The following example stops tracking a gamepad that's been removed. You'll also need to handle what happens to the gamepads that you're tracking when they're removed; for example, this code only tracks input from one gamepad, and simply sets it to `nullptr` when it's removed. You'll need to check every frame if your gamepad is active,

and update which gamepad you're gathering input from when controllers are connected and disconnected.

C++

```
Gamepad::GamepadRemoved += ref new EventHandler<Gamepad^>(Platform::Object^,
Gamepad^ args)
{
    unsigned int indexRemoved;
    critical_section::scoped_lock lock{ myLock };

    if(myGamepads->IndexOf(args, &indexRemoved))
    {
        if (m_gamepad == myGamepads->GetAt(indexRemoved))
        {
            m_gamepad = nullptr;
        }

        myGamepads->RemoveAt(indexRemoved);
    }
}
```

C#

```
Gamepad.GamepadRemoved += (object sender, Gamepad e) =>
{
    lock (myLock)
    {
        int indexRemoved = myGamepads.IndexOf(e);

        if (indexRemoved > -1)
        {
            if (mainGamepad == myGamepads[indexRemoved])
            {
                mainGamepad = null;
            }

            myGamepads.RemoveAt(indexRemoved);
        }
    }
};
```

See [Input practices for games](#) for more information.

Users and headsets

Each gamepad can be associated with a user account to link their identity to their gameplay, and can have a headset attached to facilitate voice chat or in-game features.

To learn more about working with users and headsets, see [Tracking users and their devices](#) and [Headset](#).

Reading the gamepad

After you identify the gamepad that you're interested in, you're ready to gather input from it. However, unlike some other kinds of input that you might be used to, gamepads don't communicate state-change by raising events. Instead, you take regular readings of their current state by *polling* them.

Polling the gamepad

Polling captures a snapshot of the navigation device at a precise point in time. This approach to input gathering is a good fit for most games because their logic typically runs in a deterministic loop rather than being event-driven; it's also typically simpler to interpret game commands from input gathered all at once than it is from many single inputs gathered over time.

You poll a gamepad by calling [GetCurrentReading](#); this function returns a [GamepadReading](#) that contains the state of the gamepad.

The following example polls a gamepad for its current state.

C++

```
auto gamepad = myGamepads[0];  
  
GamepadReading reading = gamepad->GetCurrentReading();
```

C#

```
Gamepad gamepad = myGamepads[0];  
  
GamepadReading reading = gamepad.GetCurrentReading();
```

In addition to the gamepad state, each reading includes a timestamp that indicates precisely when the state was retrieved. The timestamp is useful for relating to the timing of previous readings or to the timing of the game simulation.

Reading the thumbsticks

Each thumbstick provides an analog reading between -1.0 and +1.0 in the X and Y axes. In the X axis, a value of -1.0 corresponds to the left-most thumbstick position; a value of +1.0 corresponds to right-most position. In the Y axis, a value of -1.0 corresponds to the bottom-most thumbstick position; a value of +1.0 corresponds to the top-most position. In both axes, the value is approximately 0.0 when the stick is in the center position, but it's normal for the precise value to vary, even between subsequent readings; strategies for mitigating this variation are discussed later in this section.

The value of the left thumbstick's X axis is read from the `LeftThumbstickX` property of the `GamepadReading` structure; the value of the Y axis is read from the `LeftThumbstickY` property. The value of the right thumbstick's X axis is read from the `RightThumbstickX` property; the value of the Y axis is read from the `RightThumbstickY` property.

C++

```
float leftStickX = reading.LeftThumbstickX; // returns a value between
-1.0 and +1.0
float leftStickY = reading.LeftThumbstickY; // returns a value between
-1.0 and +1.0
float rightStickX = reading.RightThumbstickX; // returns a value between
-1.0 and +1.0
float rightStickY = reading.RightThumbstickY; // returns a value between
-1.0 and +1.0
```

C#

```
double leftStickX = reading.LeftThumbstickX; // returns a value between
-1.0 and +1.0
double leftStickY = reading.LeftThumbstickY; // returns a value between
-1.0 and +1.0
double rightStickX = reading.RightThumbstickX; // returns a value between
-1.0 and +1.0
double rightStickY = reading.RightThumbstickY; // returns a value between
-1.0 and +1.0
```

When reading the thumbstick values, you'll notice that they don't reliably produce a neutral reading of 0.0 when the thumbstick is at rest in the center position; instead, they'll produce different values near 0.0 each time the thumbstick is moved and returned to the center position. To mitigate these variations, you can implement a small *deadzone*, which is a range of values near the ideal center position that are ignored. One way to implement a deadzone is to determine how far from center the thumbstick has moved, and ignoring the readings that are nearer than some distance you choose. You can compute the distance roughly—it's not exact because thumbstick readings are essentially polar, not planar, values—just by using the Pythagorean theorem. This produces a radial deadzone.

The following example demonstrates a basic radial deadzone using the Pythagorean theorem.

C++

```
float leftStickX = reading.LeftThumbstickX;    // returns a value between
-1.0 and +1.0
float leftStickY = reading.LeftThumbstickY;    // returns a value between
-1.0 and +1.0

// choose a deadzone -- readings inside this radius are ignored.
const float deadzoneRadius = 0.1;
const float deadzoneSquared = deadzoneRadius * deadzoneRadius;

// Pythagorean theorem -- for a right triangle, hypotenuse^2 = (opposite
side)^2 + (adjacent side)^2
auto oppositeSquared = leftStickY * leftStickY;
auto adjacentSquared = leftStickX * leftStickX;

// accept and process input if true; otherwise, reject and ignore it.
if ((oppositeSquared + adjacentSquared) > deadzoneSquared)
{
    // input accepted, process it
}
```

C#

```
double leftStickX = reading.LeftThumbstickX;    // returns a value between
-1.0 and +1.0
double leftStickY = reading.LeftThumbstickY;    // returns a value between
-1.0 and +1.0

// choose a deadzone -- readings inside this radius are ignored.
const double deadzoneRadius = 0.1;
const double deadzoneSquared = deadzoneRadius * deadzoneRadius;

// Pythagorean theorem -- for a right triangle, hypotenuse^2 = (opposite
side)^2 + (adjacent side)^2
double oppositeSquared = leftStickY * leftStickY;
double adjacentSquared = leftStickX * leftStickX;

// accept and process input if true; otherwise, reject and ignore it.
if ((oppositeSquared + adjacentSquared) > deadzoneSquared)
{
    // input accepted, process it
}
```

Each thumbstick also acts as a button when pressed inward; for more information on reading this input, see [Reading the buttons](#).

Reading the triggers

The triggers are represented as floating-point values between 0.0 (fully released) and 1.0 (fully depressed). The value of the left trigger is read from the `LeftTrigger` property of the `GamepadReading` structure; the value of the right trigger is read from the `RightTrigger` property.

C++

```
float leftTrigger = reading.LeftTrigger; // returns a value between 0.0
and 1.0
float rightTrigger = reading.RightTrigger; // returns a value between 0.0
and 1.0
```

C#

```
double leftTrigger = reading.LeftTrigger; // returns a value between 0.0
and 1.0
double rightTrigger = reading.RightTrigger; // returns a value between 0.0
and 1.0
```

Reading the buttons

Each of the gamepad buttons—the four directions of the D-pad, left and right bumpers, left and right thumbstick press, **A**, **B**, **X**, **Y**, **View**, and **Menu**—provides a digital reading that indicates whether it's pressed (down) or released (up). For efficiency, button readings aren't represented as individual boolean values; instead, they're all packed into a single bitfield that's represented by the `GamepadButtons` enumeration.

The button values are read from the `Buttons` property of the `GamepadReading` structure. Because this property is a bitfield, bitwise masking is used to isolate the value of the button that you're interested in. The button is pressed (down) when the corresponding bit is set; otherwise, it's released (up).

The following example determines whether the A button is pressed.

C++

```
if (GamepadButtons::A == (reading.Buttons & GamepadButtons::A))
{
    // button A is pressed
}
```

C#

```
if (GamepadButtons.A == (reading.Buttons & GamepadButtons.A))
{
    // button A is pressed
}
```

The following example determines whether the A button is released.

C++

```
if (GamepadButtons::None == (reading.Buttons & GamepadButtons::A))
{
    // button A is released
}
```

C#

```
if (GamepadButtons.None == (reading.Buttons & GamepadButtons.A))
{
    // button A is released
}
```

Sometimes you might want to determine when a button transitions from pressed to released or released to pressed, whether multiple buttons are pressed or released, or if a set of buttons is arranged in a particular way—some pressed, some not. For information on how to detect each of these conditions, see [Detecting button transitions](#) and [Detecting complex button arrangements](#).

Run the gamepad input sample

The [GamepadUWP sample \(github\)](#) [↗](#) demonstrates how to connect to a gamepad and read its state.

Vibration and impulse triggers overview

The vibration motors inside a gamepad are for providing tactile feedback to the user. Games use this ability to create a greater sense of immersion, to help communicate status information (such as taking damage), to signal proximity to important objects, or for other creative uses.

Xbox One gamepads are equipped with a total of four independent vibration motors. Two are large motors located in the gamepad body; the left motor provides rough, high-amplitude vibration, while the right motor provides gentler, more subtle vibration.

The other two are small motors, one inside each trigger, that provide sharp bursts of vibration directly to the user's trigger fingers; this unique ability of the Xbox One gamepad is the reason its triggers are referred to as *impulse triggers*. By orchestrating these motors together, a wide range of tactile sensations can be produced.

Using vibration and impulse

Gamepad vibration is controlled through the [Vibration](#) property of the [Gamepad](#) class. `Vibration` is an instance of the [GamepadVibration](#) structure which is made up of four floating point values; each value represents the intensity of one of the motors.

Although the members of the `Gamepad.Vibration` property can be modified directly, it's recommended that you initialize a separate `GamepadVibration` instance to the values you want, and then copy it into the `Gamepad.Vibration` property to change the actual motor intensities all at once.

The following example demonstrates how to change the motor intensities all at once.

C++

```
// get the first gamepad
Gamepad^ gamepad = Gamepad::Gamepads->GetAt(0);

// create an instance of GamepadVibration
GamepadVibration vibration;

// ... set vibration levels on vibration struct here

// copy the GamepadVibration struct to the gamepad
gamepad.Vibration = vibration;
```

C#

```
// get the first gamepad
Gamepad gamepad = Gamepad.Gamepads[0];

// create an instance of GamepadVibration
GamepadVibration vibration = new GamepadVibration();

// ... set vibration levels on vibration struct here

// copy the GamepadVibration struct to the gamepad
gamepad.Vibration = vibration;
```

Using the vibration motors

The left and right vibration motors take floating point values between 0.0 (no vibration) and 1.0 (most intense vibration). The intensity of the left motor is set by the `LeftMotor` property of the `GamepadVibration` structure; the intensity of the right motor is set by the `RightMotor` property.

The following example sets the intensity of both vibration motors and activates gamepad vibration.

C++

```
GamepadVibration vibration;  
vibration.LeftMotor = 0.80; // sets the intensity of the left motor to 80%  
vibration.RightMotor = 0.25; // sets the intensity of the right motor to 25%  
gamepad.Vibration = vibration;
```

C#

```
GamepadVibration vibration = new GamepadVibration();  
vibration.LeftMotor = 0.80; // sets the intensity of the left motor to 80%  
vibration.RightMotor = 0.25; // sets the intensity of the right motor to 25%  
mainGamepad.Vibration = vibration;
```

Remember that these two motors are not identical so setting these properties to the same value doesn't produce the same vibration in one motor as in the other. For any value, the left motor produces a stronger vibration at a lower frequency than the right motor which—for the same value—produces a gentler vibration at a higher frequency. Even at the maximum value, the left motor can't produce the high frequencies of the right motor, nor can the right motor produce the high forces of the left motor. Still, because the motors are rigidly connected by the gamepad body, players don't experience the vibrations fully independently even though the motors have different characteristics and can vibrate with different intensities. This arrangement allows for a wider, more expressive range of sensations to be produced than if the motors were identical.

Using the impulse triggers

Each impulse trigger motor takes a floating point value between 0.0 (no vibration) and 1.0 (most intense vibration). The intensity of the left trigger motor is set by the `LeftTrigger` property of the `GamepadVibration` structure; the intensity of the right trigger is set by the `RightTrigger` property.

The following example sets intensity of both impulse triggers and activates them.

C++

```
GamepadVibration vibration;  
vibration.LeftTrigger = 0.75; // sets the intensity of the left trigger to  
75%  
vibration.RightTrigger = 0.50; // sets the intensity of the right trigger to  
50%  
gamepad.Vibration = vibration;
```

C#

```
GamepadVibration vibration = new GamepadVibration();  
vibration.LeftTrigger = 0.75; // sets the intensity of the left trigger to  
75%  
vibration.RightTrigger = 0.50; // sets the intensity of the right trigger to  
50%  
mainGamepad.Vibration = vibration;
```

Unlike the others, the two vibration motors inside the triggers are identical so they produce the same vibration in either motor for the same value. However, because these motors are not rigidly connected in any way, players experience the vibrations independently. This arrangement allows for fully independent sensations to be directed to both triggers simultaneously, and helps them to convey more specific information than the motors in the gamepad body can.

Run the gamepad vibration sample

The [GamepadVibrationUWP sample \(github\)](#) [↗] demonstrates how the gamepad vibration motors and impulse triggers are used to produce a variety of effects.

See also

- [Windows.Gaming.Input.UINavigationController](#)
- [Windows.Gaming.Input.IGameController](#)
- [Input practices for games](#)

Arcade stick

Article • 10/20/2022

This page describes the basics of programming for arcade sticks using [Windows.Gaming.Input.ArcadeStick](#) and related APIs for the Universal Windows Platform (UWP).

By reading this page, you'll learn:

- how to gather a list of connected arcade sticks and their users
- how to detect that an arcade stick has been added or removed
- how to read input from one or more arcade sticks
- how arcade sticks behave as UI navigation devices

Arcade stick overview

Arcade sticks are input devices valued for reproducing the feel of stand-up arcade machines and for their high-precision digital controls. Arcade sticks are the perfect input device for head-to-head-fighting and other arcade-style games, and are suitable for any game that works well with all-digital controls. Arcade sticks are supported in UWP apps for Windows 10 or Windows 11 and Xbox One apps through the [Windows.Gaming.Input](#) namespace.

Arcade sticks are equipped with an 8-way digital joystick, six **Action** buttons (represented as A1-A6 in the image below), and two **Special** buttons (represented as S1 and S2); they're all-digital input devices that don't support analog controls or vibration. Arcade sticks are also equipped with **View** and **Menu** buttons used to support UI navigation but they're not intended to support gameplay commands and can't be readily accessed as joystick buttons.



UI navigation

In order to ease the burden of supporting many different input devices for user interface navigation and to encourage consistency between games and devices, most *physical* input devices simultaneously act as a separate *logical* input device called a [UI navigation controller](#). The UI navigation controller provides a common vocabulary for UI navigation commands across input devices.

As a UI navigation controller, arcade sticks map the [required set](#) of navigation commands to the joystick and **View**, **Menu**, **Action 1**, and **Action 2** buttons.

Navigation command	Arcade stick input
Up	Stick up
Down	Stick down
Left	Stick left
Right	Stick right
View	View button
Menu	Menu button
Accept	Action 1 button
Cancel	Action 2 button

Arcade sticks don't map any of the [optional set](#) of navigation commands.

Detect and track arcade sticks

Detecting and tracking arcade sticks works in exactly the same way as it does for gamepads, except with the [ArcadeStick](#) class instead of the [Gamepad](#) class. See [Gamepad and vibration](#) for more information.

Reading the arcade stick

After you identify the arcade stick that you're interested in, you're ready to gather input from it. However, unlike some other kinds of input that you might be used to, arcade sticks don't communicate state-change by raising events. Instead, you take regular readings of their current state by *polling* them.

Polling the arcade stick

Polling captures a snapshot of the arcade stick at a precise point in time. This approach to input gathering is a good fit for most games because their logic typically runs in a deterministic loop rather than being event-driven; it's also typically simpler to interpret game commands from input gathered all at once than it is from many single inputs gathered over time.

You poll an arcade stick by calling [GetCurrentReading](#); this function returns an [ArcadeStickReading](#) that contains the state of the arcade stick.

The following example polls an arcade stick for its current state.

C++

```
auto arcadestick = myArcadeSticks[0];

ArcadeStickReading reading = arcadestick->GetCurrentReading();
```

In addition to the arcade stick state, each reading includes a timestamp that indicates precisely when the state was retrieved. The timestamp is useful for relating to the timing of previous readings or to the timing of the game simulation.

Reading the buttons

Each of the arcade stick buttons—the four directions of the joystick, six **Action** buttons, and two **Special** buttons—provides a digital reading that indicates whether it's pressed (down) or released (up). For efficiency, button readings aren't represented as individual boolean values; instead, they're all packed into a single bitfield that's represented by the [ArcadeStickButtons](#) enumeration.

📌 Note

Arcade sticks are equipped with additional buttons used for UI navigation such as the **View** and **Menu** buttons. These buttons are not a part of the [ArcadeStickButtons](#) enumeration and can only be read by accessing the arcade stick as a UI navigation device. For more information, see [UI Navigation Device](#).

The button values are read from the `Buttons` property of the [ArcadeStickReading](#) structure. Because this property is a bitfield, bitwise masking is used to isolate the value of the button that you're interested in. The button is pressed (down) when the corresponding bit is set; otherwise, it's released (up).

The following example determines whether the **Action 1** button is pressed.

C++

```
if (ArcadeStickButtons::Action1 == (reading.Buttons &
ArcadeStickButtons::Action1))
{
    // Action 1 is pressed
}
```

The following example determines whether the **Action 1** button is released.

C++

```
if (ArcadeStickButtons::None == (reading.Buttons &
ArcadeStickButtons::Action1))
{
    // Action 1 is released (not pressed)
}
```

Sometimes you might want to determine when a button transitions from pressed to released or released to pressed, whether multiple buttons are pressed or released, or if a set of buttons are arranged in a particular way—some pressed, some not. For information on how to detect these conditions, see [Detecting button transitions](#) and [Detecting complex button arrangements](#).

Run the InputInterfacing sample

The [InputInterfacingUWP sample \(github\)](#) [↗](#) demonstrates how to use arcade sticks and different kinds of input devices in tandem, as well as how these input devices behave as UI navigation controllers.

See also

- [Windows.Gaming.Input.UINavigationController](#)
- [Windows.Gaming.Input.IGameController](#)
- [Input practices for games](#)

Racing wheel and force feedback

Article • 05/02/2023

This page describes the basics of programming for racing wheels on Xbox One using [Windows.Gaming.Input.RacingWheel](#) and related APIs for the Universal Windows Platform (UWP).

By reading this page, you'll learn:

- how to gather a list of connected racing wheels and their users
- how to detect that a racing wheel has been added or removed
- how to read input from one or more racing wheels
- how to send force feedback commands
- how racing wheels behave as UI navigation devices

Racing wheel overview

Racing wheels are input devices that resemble the feel of a real racecar cockpit. Racing wheels are the perfect input device for both arcade-style and simulation-style racing games that feature cars or trucks. Racing wheels are supported in Windows 10 or Windows 11 and Xbox One UWP apps by the [Windows.Gaming.Input](#) namespace.

Racing wheels are offered at a variety of price points, generally having more and better input and force feedback capabilities as their price points rise. All racing wheels are equipped with an analog steering wheel, analog throttle and brake controls, and some on-wheel buttons. Some racing wheels are additionally equipped with analog clutch and handbrake controls, pattern shifters, and force feedback capabilities. Not all racing wheels are equipped with the same sets of features, and may also vary in their support for certain features—for example, steering wheels might support different ranges of rotation and pattern shifters might support different numbers of gears.

Device capabilities

Different racing wheels offer different sets of optional device capabilities and varying levels of support for those capabilities; this level of variation between a single kind of input device is unique among the devices supported by the [Windows.Gaming.Input](#) API. Furthermore, most devices you'll encounter will support at least some optional capabilities or other variations. Because of this, it's important to determine the capabilities of each connected racing wheel individually and to support the full variation of capabilities that makes sense for your game.

For more information, see [Determining racing wheel capabilities](#).

Force feedback

Some racing wheels offer true force feedback—that is, they can apply actual forces on an axis of control such as their steering wheel—not just simple vibration. Games use this ability to create a greater sense of immersion (*simulated crash damage*, "road feel") and to increase the challenge of driving well.

For more information, see [Force feedback overview](#).

UI navigation

In order to ease the burden of supporting the different input devices for user interface navigation and to encourage consistency between games and devices, most *physical* input devices simultaneously act as a separate *logical* input device called a [UI navigation controller](#). The UI navigation controller provides a common vocabulary for UI navigation commands across input devices.

Due to their unique focus on analog controls and the degree of variation between different racing wheels, they're typically equipped with a digital D-pad, **View**, **Menu**, **A**, **B**, **X**, and **Y** buttons that resemble those of a [gamepad](#); these buttons aren't intended to support gameplay commands and can't be readily accessed as racing wheel buttons.

As a UI navigation controller, racing wheels map the [required set](#) of navigation commands to the left thumbstick, D-pad, **View**, **Menu**, **A**, and **B** buttons.

Navigation command	Racing wheel input
Up	D-pad up
Down	D-pad down
Left	D-pad left
Right	D-pad right
View	View button
Menu	Menu button
Accept	A button
Cancel	B button

Additionally, some racing wheels might map some of the [optional set](#) of navigation commands to other inputs they support, but command mappings can vary from device to device. Consider supporting these commands as well, but make sure that these commands are not essential to navigating your game's interface.

Navigation command	Racing wheel input
Page Up	<i>varies</i>
Page Down	<i>varies</i>
Page Left	<i>varies</i>
Page Right	<i>varies</i>
Scroll Up	<i>varies</i>
Scroll Down	<i>varies</i>
Scroll Left	<i>varies</i>
Scroll Right	<i>varies</i>
Context 1	X Button (<i>commonly</i>)
Context 2	Y Button (<i>commonly</i>)
Context 3	<i>varies</i>
Context 4	<i>varies</i>

Detect and track racing wheels

Detecting and tracking racing wheels works in exactly the same way as it does for gamepads, except with the [RacingWheel](#) class instead of the [Gamepad](#) class. See [Gamepad and vibration](#) for more information.

Reading the racing wheel

After you identify the racing wheels that you're interested in, you're ready to gather input from them. However, unlike some other kinds of input that you might be used to, racing wheels don't communicate state-change by raising events. Instead, you take regular readings of their current states by *polling* them.

Polling the racing wheel

Polling captures a snapshot of the racing wheel at a precise point in time. This approach to input gathering is a good fit for most games because their logic typically runs in a deterministic loop rather than being event-driven; it's also typically simpler to interpret game commands from input gathered all at once than it is from many single inputs gathered over time.

You poll a racing wheel by calling [GetCurrentReading](#); this function returns a [RacingWheelReading](#) that contains the state of the racing wheel.

The following example polls a racing wheel for its current state.

C++

```
auto racingwheel = myRacingWheels[0];

RacingWheelReading reading = racingwheel->GetCurrentReading();
```

In addition to the racing wheel state, each reading includes a timestamp that indicates precisely when the state was retrieved. The timestamp is useful for relating to the timing of previous readings or to the timing of the game simulation.

Determining racing wheel capabilities

Many of the racing wheel controls are optional or support different variations even in the required controls, so you have to determine the capabilities of each racing wheel individually before you can process the input gathered in each reading of the racing wheel.

The optional controls are the handbrake, clutch, and pattern shifter; you can determine whether a connected racing wheel supports these controls by reading the [HasHandbrake](#), [HasClutch](#), and [HasPatternShifter](#) properties of the racing wheel, respectively. The control is supported if the value of the property is **true**; otherwise it's not supported.

C++

```
if (racingwheel->HasHandbrake)
{
    // the handbrake is supported
}

if (racingwheel->HasClutch)
{
    // the clutch is supported
}
```



```
if (racingwheel->HasPatternShifter)
{
    // the pattern shifter is supported
}
```

Additionally, the controls that may vary are the steering wheel and pattern shifter. The steering wheel can vary by the degree of physical rotation that the actual wheel can support, while the pattern shifter can vary by the number of distinct forward gears it supports. You can determine the greatest angle of rotation the actual wheel supports by reading the `MaxWheelAngle` property of the racing wheel; its value is the maximum supported physical angle in degrees clock-wise (positive) which is likewise supported in the counter-clock-wise direction (negative degrees). You can determine the greatest forward gear the pattern shifter supports by reading the `MaxPatternShifterGear` property of the racing wheel; its value is the highest forward gear supported, inclusive—that is, if its value is 4, then the pattern shifter supports reverse, neutral, first, second, third, and fourth gears.

C++

```
auto maxWheelDegrees = racingwheel->MaxWheelAngle;
auto maxShifterGears = racingwheel->MaxPatternShifterGear;
```

Finally, some racing wheels support force feedback through the steering wheel. You can determine whether a connected racing wheel supports force feedback by reading the `WheelMotor` property of the racing wheel. Force feedback is supported if `WheelMotor` is not `null`; otherwise it's not supported.

C++

```
if (racingwheel->WheelMotor != nullptr)
{
    // force feedback is supported
}
```

For information on how to use the force feedback capability of racing wheels that support it, see [Force feedback overview](#).

Reading the buttons

Each of the racing wheel buttons—the four directions of the D-pad, the **Previous Gear** and **Next Gear** buttons, and 16 additional buttons—provides a digital reading that indicates whether it's pressed (down) or released (up). For efficiency, button readings

aren't represented as individual boolean values; instead they're all packed into a single bitfield that's represented by the [RacingWheelButtons](#) enumeration.

❗ Note

Racing wheels are equipped with additional buttons used for UI navigation such as the **View** and **Menu** buttons. These buttons are not a part of the `RacingWheelButtons` enumeration and can only be read by accessing the racing wheel as a UI navigation device. For more information, see [UI Navigation Device](#).

The button values are read from the `Buttons` property of the [RacingWheelReading](#) structure. Because this property is a bitfield, bitwise masking is used to isolate the value of the button that you're interested in. The button is pressed (down) when the corresponding bit is set; otherwise, it's released (up).

The following example determines whether the **Next Gear** button is pressed.

C++

```
if (RacingWheelButtons::NextGear == (reading.Buttons &
RacingWheelButtons::NextGear))
{
    // Next Gear is pressed
}
```

The following example determines whether the Next Gear button is released.

C++

```
if (RacingWheelButtons::None == (reading.Buttons &
RacingWheelButtons::NextGear))
{
    // Next Gear is released (not pressed)
}
```

Sometimes you might want to determine when a button transitions from pressed to released or released to pressed, whether multiple buttons are pressed or released, or if a set of buttons are arranged in a particular way—some pressed, some not. For information on how to detect these conditions, see [Detecting button transitions](#) and [Detecting complex button arrangements](#).

Reading the wheel

The steering wheel is a required control that provides an analog reading between -1.0 and +1.0. A value of -1.0 corresponds to the left-most wheel position; a value of +1.0 corresponds to the right-most position. The value of the steering wheel is read from the `Wheel` property of the [RacingWheelReading](#) structure.

C++

```
float wheel = reading.Wheel; // returns a value between -1.0 and +1.0.
```

Although wheel readings correspond to different degrees of physical rotation in the actual wheel depending on the range of rotation supported by the physical racing wheel, you don't usually want to scale the wheel readings; wheels that support greater degrees of rotation just provide greater precision.

Reading the throttle and brake

The throttle and brake are required controls that each provide analog readings between 0.0 (fully released) and 1.0 (fully pressed) represented as floating-point values. The value of the throttle control is read from the `Throttle` property of the [RacingWheelReading](#) struct; the value of the brake control is read from the `Brake` property.

C++

```
float throttle = reading.Throttle; // returns a value between 0.0 and 1.0
float brake    = reading.Brake;    // returns a value between 0.0 and 1.0
```

Reading the handbrake and clutch

The handbrake and clutch are optional controls that each provide analog readings between 0.0 (fully released) and 1.0 (fully engaged) represented as floating-point values. The value of the handbrake control is read from the `Handbrake` property of the [RacingWheelReading](#) struct; the value of the clutch control is read from the `Clutch` property.

C++

```
float handbrake = 0.0;
float clutch    = 0.0;

if(racingwheel->HasHandbrake)
{
    handbrake = reading.Handbrake; // returns a value between 0.0 and 1.0
}
```

```
if(racingwheel->HasClutch)
{
    clutch = reading.Clutch;           // returns a value between 0.0 and 1.0
}
```

Reading the pattern shifter

The pattern shifter is an optional control that provides a digital reading between -1 and [MaxPatternShifterGear](#) represented as a signed integer value. A value of -1 or 0 correspond to the *reverse* and *neutral* gears, respectively; increasingly positive values correspond to greater forward gears up to **MaxPatternShifterGear**, inclusive. The value of the pattern shifter is read from the [PatternShifterGear](#) property of the [RacingWheelReading](#) struct.

C++

```
if (racingwheel->HasPatternShifter)
{
    gear = reading.PatternShifterGear;
}
```

ⓘ Note

The pattern shifter, where supported, exists alongside the required **Previous Gear** and **Next Gear** buttons which also affect the current gear of the player's car. A simple strategy for unifying these inputs where both are present is to ignore the pattern shifter (and clutch) when a player chooses an automatic transmission for their car, and to ignore the **Previous** and **Next Gear** buttons when a player chooses a manual transmission for their car only if their racing wheel is equipped with a pattern shifter control. You can implement a different unification strategy if this isn't suitable for your game.

Run the InputInterfacing sample

The [InputInterfacingUWP](#) [↗](#) sample app on GitHub demonstrates how to use racing wheels and different kinds of input devices in tandem; as well as how these input devices behave as UI navigation controllers.

Force feedback overview

Many racing wheels have force feedback capability to provide a more immersive and challenging driving experience. Racing wheels that support force feedback are typically equipped with a single motor that applies force to the steering wheel along a single axis, the axis of wheel rotation. Force feedback is supported in Windows 10 or Windows 11 and Xbox One UWP apps through the [Windows.Gaming.Input.ForceFeedback](#) namespace.

ⓘ Note

The force feedback APIs are capable of supporting several axes of force, but no racing wheel currently supports any feedback axis other than that of wheel rotation.

Using force feedback

These sections describe the basics of programming force feedback effects for racing wheels. Feedback is applied using effects, which are first loaded onto the force feedback device and then can be started, paused, resumed, and stopped in a manner similar to sound effects; however, you must first determine the feedback capabilities of the racing wheel.

Determining force feedback capabilities

You can determine whether a connected racing wheel supports force feedback by reading the [WheelMotor](#) property of the racing wheel. Force feedback isn't supported if `WheelMotor` is `null`; otherwise, force feedback is supported and you can proceed to determine the specific feedback capabilities of the motor, such as the axes it can affect.

C++

```
if (racingwheel->WheelMotor != nullptr)
{
    auto axes = racingwheel->WheelMotor->SupportedAxes;

    if (ForceFeedbackEffectAxes::X == (axes & ForceFeedbackEffectAxes::X))
    {
        // Force can be applied through the X axis
    }

    if (ForceFeedbackEffectAxes::Y == (axes & ForceFeedbackEffectAxes::Y))
    {
        // Force can be applied through the Y axis
    }

    if (ForceFeedbackEffectAxes::Z == (axes & ForceFeedbackEffectAxes::Z))
```

```

    {
        // Force can be applied through the Z axis
    }
}

```

Loading force feedback effects

Force feedback effects are loaded onto the feedback device where they are "played" autonomously at the command of your game. A number of basic effects are provided; custom effects can be created through a class that implements the [IForceFeedbackEffect](#) interface.

Effect class	Effect description
ConditionForceEffect	An effect that applies variable force in response to current sensor within the device.
ConstantForceEffect	An effect that applies constant force along a vector.
PeriodicForceEffect	An effect that applies variable force defined by a waveform, along a vector.
RampForceEffect	An effect that applies a linearly increasing/decreasing force along a vector.

C++

```

using FFLoadEffectResult = ForceFeedback::ForceFeedbackLoadEffectResult;

auto effect = ref new
Windows.Gaming::Input::ForceFeedback::ConstantForceEffect();
auto time = TimeSpan(10000);

effect->SetParameters(Windows::Foundation::Numerics::float3(1.0f, 0.0f,
0.0f), time);

// Here, we assume 'racingwheel' is valid and supports force feedback

IAsyncOperation<FFLoadEffectResult>^ request
    = racingwheel->WheelMotor->LoadEffectAsync(effect);

auto loadEffectTask = Concurrency::create_task(request);

loadEffectTask.then([this](FFLoadEffectResult result)
{
    if (FFLoadEffectResult::Succeeded == result)
    {
        // effect successfully loaded
    }
    else
    {

```

```
        // effect failed to load
    }
}).wait();
```

Using force feedback effects

Once loaded, effects can all be started, paused, resumed, and stopped synchronously by calling functions on the `WheelMotor` property of the racing wheel, or individually by calling functions on the feedback effect itself. Typically, you should load all the effects that you want to use onto the feedback device before gameplay begins and then use their respective `SetParameters` functions to update the effects as gameplay progresses.

C++

```
if (ForceFeedbackEffectState::Running == effect->State)
{
    effect->Stop();
}
else
{
    effect->Start();
}
```

Finally, you can asynchronously enable, disable, or reset the entire force feedback system on a particular racing wheel whenever you need.

See also

- [Windows.Gaming.Input.UINavigationController](#)
- [Windows.Gaming.Input.IGameController](#)
- [Input practices for games](#)

Flight stick

Article • 10/20/2022

This page describes the basics of programming for Xbox One-certified flight sticks using [Windows.Gaming.Input.FlightStick](#) and related APIs for the Universal Windows Platform (UWP).

By reading this page, you'll learn:

- how to gather a list of connected flight sticks and their users
- how to detect that a flight stick has been added or removed
- how to read input from one or more flight sticks
- how flight sticks behave as UI navigation devices

Overview

Flight sticks are gaming input devices that are valued for reproducing the feel of flight sticks that would be found in a plane or spaceship's cockpit. They're the perfect input device for quick and accurate control of flight. Flight sticks are supported in Windows 10 or Windows 11 and Xbox One apps through the [Windows.Gaming.Input](#) namespace.

Xbox One-certified flight sticks are equipped with the following controls:

- A twistable analog joystick capable of roll, pitch, and yaw
- An analog throttle
- Two fire buttons
- An 8-way digital hat switch
- **View** and **Menu** buttons

ⓘ Note

The **View** and **Menu** buttons are used to support UI navigation, not gameplay commands, and therefore can't be readily accessed as joystick buttons.

UI navigation

In order to ease the burden of supporting the different input devices for user interface navigation and to encourage consistency between games and devices, most *physical* input devices simultaneously act as separate *logical* input devices called [UI navigation](#)

[controllers](#). The UI navigation controller provides a common vocabulary for UI navigation commands across input devices.

As a UI navigation controller, a flight stick maps the [required set](#) of navigation commands to the joystick and **View**, **Menu**, **FirePrimary**, and **FireSecondary** buttons.

Navigation command	Flight stick input
Up	Joystick up
Down	Joystick down
Left	Joystick left
Right	Joystick right
View	View button
Menu	Menu button
Accept	FirePrimary button
Cancel	FireSecondary button

Flight sticks don't map any of the [optional set](#) of navigation commands.

Detect and track flight sticks

Detecting and tracking flight sticks works in exactly the same way as it does for gamepads, except with the [FlightStick](#) class instead of the [Gamepad](#) class. See [Gamepad and vibration](#) for more information.

Reading the flight stick

After you identify the flight stick that you're interested in, you're ready to gather input from it. However, unlike some other kinds of input that you might be used to, flight sticks don't communicate state-change by raising events. Instead, you take regular readings of their current state by *polling* them.

Polling the flight stick

Polling captures a snapshot of the flight stick at a precise point in time. This approach to input gathering is a good fit for most games because their logic typically runs in a deterministic loop rather than being event-driven. It's also typically simpler to interpret

game commands from input gathered all at once than it is from many single inputs gathered over time.

You poll a flight stick by calling [FlightStick.GetCurrentReading](#). This function returns a [FlightStickReading](#) that contains the state of the flight stick.

The following example polls a flight stick for its current state:

C++

```
auto flightStick = myFlightSticks->GetAt(0);  
FlightStickReading reading = flightStick->GetCurrentReading();
```

In addition to the flight stick state, each reading includes a timestamp that indicates precisely when the state was retrieved. The timestamp is useful for relating to the timing of previous readings or to the timing of the game simulation.

Reading the joystick and throttle input

The joystick provides an analog reading between -1.0 and 1.0 in the X, Y, and Z axes (roll, pitch, and yaw, respectively). For roll, a value of -1.0 corresponds to the left-most joystick position, while a value of 1.0 corresponds to the right-most position. For pitch, a value of -1.0 corresponds to the bottom-most joystick position, while a value of 1.0 corresponds to the top-most position. For yaw, a value of -1.0 corresponds to the most counterclockwise, twisted position, while a value of 1.0 corresponds to the most clockwise position.

In all axes, the value is approximately 0.0 when the joystick is in the center position, but it's normal for the precise value to vary, even between subsequent readings. Strategies for mitigating this variation are discussed later in this section.

The value of the joystick's roll is read from the [FlightStickReading.Roll](#) property, the value of the pitch is read from the [FlightStickReading.Pitch](#) property, and the value of the yaw is read from the [FlightStickReading.Yaw](#) property:

C++

```
// Each variable will contain a value between -1.0 and 1.0.  
float roll = reading.Roll;  
float pitch = reading.Pitch;  
float yaw = reading.Yaw;
```

When reading the joystick values, you'll notice that they don't reliably produce a neutral reading of 0.0 when the joystick is at rest in the center position; instead, they'll produce

different values near 0.0 each time the joystick is moved and returned to the center position. To mitigate these variations, you can implement a small *deadzone*, which is a range of values near the ideal center position that are ignored.

One way to implement a deadzone is to determine how far the joystick has moved from the center, and ignore readings that are nearer than some distance you choose. You can compute the distance roughly—it's not exact because joystick readings are essentially polar, not planar, values—just by using the Pythagorean theorem. This produces a radial deadzone.

The following example demonstrates a basic radial deadzone using the Pythagorean theorem:

C++

```
// Choose a deadzone. Readings inside this radius are ignored.
const float deadzoneRadius = 0.1f;
const float deadzoneSquared = deadzoneRadius * deadzoneRadius;

// Pythagorean theorem: For a right triangle, hypotenuse^2 = (opposite
side)^2 + (adjacent side)^2
float oppositeSquared = pitch * pitch;
float adjacentSquared = roll * roll;

// Accept and process input if true; otherwise, reject and ignore it.
if ((oppositeSquared + adjacentSquared) < deadzoneSquared)
{
    // Input accepted, process it.
}
```

Reading the buttons and hat switch

Each of the flight stick's two fire buttons provides a digital reading that indicates whether it's pressed (down) or released (up). For efficiency, button readings aren't represented as individual boolean values—instead, they're all packed into a single bitfield that's represented by the [FlightStickButtons](#) enumeration. In addition, the 8-way hat switch provides a direction packed into a single bitfield that's represented by the [GameControllerSwitchPosition](#) enumeration.

❗ Note

Flight sticks are equipped with additional buttons used for UI navigation such as the **View** and **Menu** buttons. These buttons are not part of the `FlightStickButtons`

enumeration and can only be read by accessing the flight stick as a UI navigation device. For more information, see [UI navigation controller](#).

The button values are read from the [FlightStickReading.Buttons](#) property. Because this property is a bitfield, bitwise masking is used to isolate the value of the button that you're interested in. The button is pressed (down) when the corresponding bit is set; otherwise, it's released (up).

The following example determines whether the **FirePrimary** button is pressed:

C++

```
if (FlightStickButtons::FirePrimary == (reading.Buttons &
FlightStickButtons::FirePrimary))
{
    // FirePrimary is pressed.
}
```

The following example determines whether the **FirePrimary** button is released:

C++

```
if (FlightStickButtons::None == (reading.Buttons &
FlightStickButtons::FirePrimary))
{
    // FirePrimary is released (not pressed).
}
```

Sometimes you might want to determine when a button transitions from pressed to released or released to pressed, whether multiple buttons are pressed or released, or if a set of buttons are arranged in a particular way—some pressed, some not. For information on how to detect each of these conditions, see [Detecting button transitions](#) and [Detecting complex button arrangements](#).

The hat switch value is read from the [FlightStickReading.HatSwitch](#) property. Because this property is also a bitfield, bitwise masking is again used to isolate the position of the hat switch.

The following example determines whether the hat switch is in the up position:

C++

```
if (GameControllerSwitchPosition::Up == (reading.HatSwitch &
GameControllerSwitchPosition::Up))
{
```

```
    // The hat switch is in the up position.  
}
```

The following example determines if the hat switch is in the center position:

C++

```
if (GameControllerSwitchPosition::Center == (reading.HatSwitch &  
GameControllerSwitchPosition::Center))  
{  
    // The hat switch is in the center position.  
}
```

See also

- [Windows.Gaming.Input.UINavigationController class](#)
- [Windows.Gaming.Input.IGameController interface](#)
- [Input practices for games](#)

Raw game controller

Article • 10/20/2022

This page describes the basics of programming for nearly any type of game controller using [Windows.Gaming.Input.RawGameController](#) and related APIs for the Universal Windows Platform (UWP).

By reading this page, you'll learn:

- how to gather a list of connected raw game controllers and their users
- how to detect that a raw game controller has been added or removed
- how to get the capabilities of a raw game controller
- how to read input from a raw game controller

Overview

A raw game controller is a generic representation of a game controller, with inputs found on many different kinds of common game controllers. These inputs are exposed as simple arrays of unnamed buttons, switches, and axes. Using a raw game controller, you can allow customers to create custom input mappings no matter what type of controller they're using.

The [RawGameController](#) class is really meant for scenarios when the other input classes ([ArcadeStick](#), [FlightStick](#), and so on) don't meet your needs—if you want something more generic, anticipating that customers will use many different types of game controllers, then this class is for you.

Detect and track raw game controllers

Detecting and tracking raw game controllers works in exactly the same way as it does for gamepads, except with the [RawGameController](#) class instead of the [Gamepad](#) class. See [Gamepad and vibration](#) for more information.

Get the capabilities of a raw game controller

After you identify the raw game controller that you're interested in, you can gather information on the capabilities of the controller. You can get the number of buttons on the raw game controller with [RawGameController.ButtonCount](#), the number of analog axes with [RawGameController.AxisCount](#), and the number of switches with

[RawGameController.SwitchCount](#). Additionally, you can get the type of a switch using [RawGameController.GetSwitchKind](#).

The following example gets the input counts of a raw game controller:

C++

```
auto rawGameController = myRawGameControllers->GetAt(0);
int buttonCount = rawGameController->ButtonCount;
int axisCount = rawGameController->AxisCount;
int switchCount = rawGameController->SwitchCount;
```

The following example determines the type of each switch:

C++

```
for (uint32_t i = 0; i < switchCount; i++)
{
    GameControllerSwitchKind mySwitch = rawGameController->GetSwitchKind(i);
}
```

Reading the raw game controller

After you know the number of inputs on a raw game controller, you're ready to gather input from it. However, unlike some other kinds of input that you might be used to, a raw game controller doesn't communicate state-change by raising events. Instead, you take regular readings of its current state by *polling* it.

Polling the raw game controller

Polling captures a snapshot of the raw game controller at a precise point in time. This approach to input gathering is a good fit for most games because their logic typically runs in a deterministic loop rather than being event-driven. It's also typically simpler to interpret game commands from input gathered all at once than it is from many single inputs gathered over time.

You poll a raw game controller by calling [RawGameController.GetCurrentReading](#). This function populates arrays for buttons, switches, and axes that contain the state of the raw game controller.

The following example polls a raw game controller for its current state:

C++

```
Platform::Array<bool>^ currentButtonReading =
    ref new Platform::Array<bool>(buttonCount);

Platform::Array<GameControllerSwitchPosition>^ currentSwitchReading =
    ref new Platform::Array<GameControllerSwitchPosition>(switchCount);

Platform::Array<double>^ currentAxisReading = ref new
Platform::Array<double>(axisCount);

rawGameController->GetCurrentReading(
    currentButtonReading,
    currentSwitchReading,
    currentAxisReading);
```

There is no guarantee of which position in each array will hold which input value among different types of controllers, so you'll need to check which input is which using the methods [RawGameController.GetButtonLabel](#) and [RawGameController.GetSwitchKind](#).

GetButtonLabel will tell you the text or symbol that's printed on the physical button, rather than the button's function—therefore, it's best used as an aid for UI for cases where you want to give the player hints about which buttons perform which functions. **GetSwitchKind** will tell you the type of switch (that is, how many positions it has), but not the name.

There is no standardized way to get the label of an axis or switch, so you'll need to test these yourself to determine which input is which.

If you have a specific controller that you want to support, you can get the [RawGameController.HardwareProductId](#) and [RawGameController.HardwareVendorId](#) and check if they match that controller. The position of each input in each array is the same for every controller with the same **HardwareProductId** and **HardwareVendorId**, so you don't have to worry about your logic potentially being inconsistent among different controllers of the same type.

In addition to the raw game controller state, each reading returns a timestamp that indicates precisely when the state was retrieved. The timestamp is useful for relating to the timing of previous readings or to the timing of the game simulation.

Reading the buttons and switches

Each of the raw game controller's buttons provides a digital reading that indicates whether it's pressed (down) or released (up). Button readings are represented as individual Boolean values in a single array. The label for each button can be found using

[RawGameController.GetButtonLabel](#) with the index of the Boolean value in the array. Each value is represented as a [GameControllerButtonLabel](#).

The following example determines whether the **XboxA** button is pressed:

C++

```
for (uint32_t i = 0; i < buttonCount; i++)
{
    if (currentButtonReading[i])
    {
        GameControllerButtonLabel buttonLabel = rawGameController->GetButtonLabel(i);

        if (buttonLabel == GameControllerButtonLabel::XboxA)
        {
            // XboxA is pressed.
        }
    }
}
```

Sometimes you might want to determine when a button transitions from pressed to released or released to pressed, whether multiple buttons are pressed or released, or if a set of buttons are arranged in a particular way—some pressed, some not. For information on how to detect each of these conditions, see [Detecting button transitions](#) and [Detecting complex button arrangements](#).

Switch values are provided as an array of [GameControllerSwitchPosition](#). Because this property is a bitfield, bitwise masking is used to isolate the direction of the switch.

The following example determines whether each switch is in the up position:

C++

```
for (uint32_t i = 0; i < switchCount; i++)
{
    if (GameControllerSwitchPosition::Up ==
        (currentSwitchReading[i] & GameControllerSwitchPosition::Up))
    {
        // The switch is in the up position.
    }
}
```

Reading the analog inputs (sticks, triggers, throttles, and so on)

An analog axis provides a reading between 0.0 and 1.0. This includes each dimension on a stick such as X and Y for standard sticks or even X, Y, and Z axes (roll, pitch, and yaw, respectively) for flight sticks.

The values can represent analog triggers, throttles, or any other type of analog input. These values are not provided with labels, so we suggest that your code is tested with a variety of input devices to ensure that they match correctly with your assumptions.

In all axes, the value is approximately 0.5 for a stick when it is in the center position, but it's normal for the precise value to vary, even between subsequent readings; strategies for mitigating this variation are discussed later in this section.

The following example shows how to read the analog values from an Xbox controller:

C++

```
// Xbox controllers have 6 axes: 2 for each stick and one for each trigger.  
float leftStickX = currentAxisReading[0];  
float leftStickY = currentAxisReading[1];  
float rightStickX = currentAxisReading[2];  
float rightStickY = currentAxisReading[3];  
float leftTrigger = currentAxisReading[4];  
float rightTrigger = currentAxisReading[5];
```

When reading the stick values, you'll notice that they don't reliably produce a neutral reading of 0.5 when at rest in the center position; instead, they'll produce different values near 0.5 each time the stick is moved and returned to the center position. To mitigate these variations, you can implement a small *deadzone*, which is a range of values near the ideal center position that are ignored.

One way to implement a deadzone is to determine how far the stick has moved from the center, and ignore readings that are nearer than some distance you choose. You can compute the distance roughly—it's not exact because stick readings are essentially polar, not planar, values—just by using the Pythagorean theorem. This produces a radial deadzone.

The following example demonstrates a basic radial deadzone using the Pythagorean theorem:

C++

```
// Choose a deadzone. Readings inside this radius are ignored.  
const float deadzoneRadius = 0.1f;  
const float deadzoneSquared = deadzoneRadius * deadzoneRadius;  
  
// Pythagorean theorem: For a right triangle, hypotenuse^2 = (opposite
```

```
side)^2 + (adjacent side)^2
float oppositeSquared = leftStickY * leftStickY;
float adjacentSquared = leftStickX * leftStickX;

// Accept and process input if true; otherwise, reject and ignore it.
if ((oppositeSquared + adjacentSquared) < deadzoneSquared)
{
    // Input accepted, process it.
}
```

See also

- [Input for games](#)
- [Input practices for games](#)
- [Windows.Gaming.Input namespace](#)
- [Windows.Gaming.Input.RawGameController class](#)
- [Windows.Gaming.Input.IGameController interface](#)
- [Windows.Gaming.Input.IGameControllerBatteryInfo interface](#)

Headset

Article • 10/20/2022

This page describes the basics of programming for headsets using [Windows.Gaming.Input.Headset](#) and related APIs for the Universal Windows Platform (UWP).

By reading this page, you'll learn:

- How to access a headset that's connected to an input or navigation device
- How to detect that a headset has been connected or disconnected

Headset overview

Headsets are audio capture and playback devices most often used to communicate with other players in online games but can also be used in gameplay or for other creative uses. Headsets are supported in Windows 10 or Windows 11 and Xbox UWP apps through the [Windows.Gaming.Input](#) namespace.

Detect and track headsets

Headsets are managed by the system, therefore you don't have to create or initialize them. The system provides access to a headset through the input device its connected to and events to notify you when a headset is connected or disconnected.

IGameController.Headset

All input devices in the [Windows.Gaming.Input](#) namespace implement the [IGameController](#) interface which defines the [Headset](#) property to be the headset currently connected to the device.

Connecting and disconnecting headsets.

When a headset is connected or disconnected, the [HeadsetConnected](#) and [HeadsetDisconnected](#) events are raised. You can register handlers for these events to keep track of whether an input device currently has a headset connected to it.

The following example shows how to register a handler for the `HeadsetConnected` event.

```
C++
```

```

auto inputDevice = myGamepads[0]; // or arcade stick, racing wheel

inputDevice.HeadsetConnected += ref new TypedEventHandler<IGameController^,
Headset^>(IGameController^ device, Headset^ headset)
{
    // enable headset capture and playback on this device
}

```

The following example shows how to register a handler for the `HeadsetDisconnected` event.

C++

```

auto inputDevice = myGamepads[0]; // or arcade stick, racing wheel

inputDevice.HeadsetDisconnected += ref new
TypedEventHandler<IGameController^, Headset^>(IGameController^ device,
Headset^ headset)
{
    // disable headset capture and playback on this device
}

```

Using the headset

The `Headset` class is made up of two strings that represent XAudio endpoint IDs--one for audio capture (recording from the headset microphone) and one for audio rendering (playback through the headset earpiece).

The details of working with XAudio are not discussed here, for more information see the [XAudio2 programming guide](#) and [XAudio2 API reference](#).